

Algorithms (4190.407)

Homework 5-2

Minimim Spanning Tree Algorithms

Yeonjun Kim
kyjun0803@snu.ac.kr

November 20, 2025

1 Introduction

In this assignment, I implemented Kruskal and Prim algorithms which are used to find MST(Minimum Spanning Tree)s. Also, I implemented the heap and union-find data structures from scratch.

How to run

Before running the program, you should place 'input.txt' in this directory. In 'input.txt', you write N, M, and the graph edges. You can run the program by `$ python3 202413755_Task5.py`, and see the results in `output_kruskal.txt`, `output_prim.txt`, and `output_prim_fibo.txt`.

2 Time Complexity Analysis

Theoretical comparison

The theoretical time complexity of each of the three algorithms are shown in Table 1.

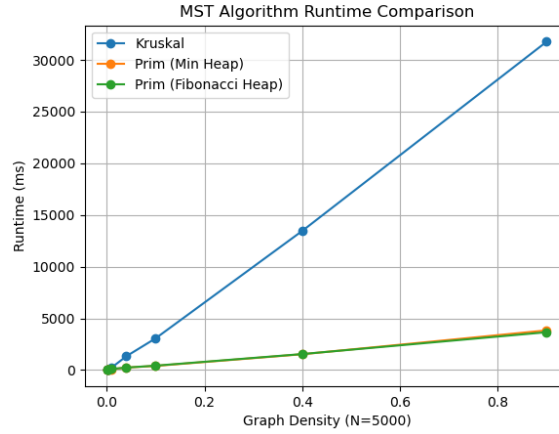
	Kruskal	Prim (w. binary heap)	Prim (w. Fibonacci heap)
Time Complexity	$O(E \log V)$	$O(E \log V)$	$O(E + V \log V)$

Table 1: Theoretical time complexity comparison of Kruskal, Prim and Prim with Fibonacci heap algorithms

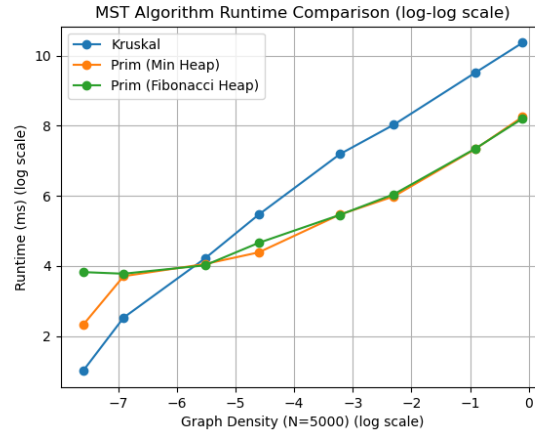
Experimental settings

- To experiment with the time complexity of each algorithm, I randomly generated undirected, connected, simple graphs with number of nodes $N = 5000$.
- I changed the density of the random graph starting from 0.0005 to 0.9.

Results



(a) running time to graph density



(b) running time to graph density, in log scale

Figure 1: Timing experiment result of Kruskal, Prim with binary Heap, and Prim with Fibonacci heap.

We can compare the performance of each algorithm at varying densities in 1b. In sparse graphs ($density < 0.01$), Kruskal outperformed Prim algorithm. However, as the density grew, Prim algorithm showed faster performance. In the most dense graph $density = 0.9$,

	Kruskal	Prim (w. binary heap)	Prim (w. Fibonacci heap)
Time Consumed (seconds)	12.57	58.50	25.39
Memory used (megabytes)	570.35	567.01	573.81

Table 2: Scaling challenge results for three algorithms.

Prim algorithm with Fibonacci heap was slightly faster than Prim implemented with a binary heap.

3 Scaling Challenge

Experimental settings

- To test the performance the algorithms in very large (but sparse) connected graphs, I testing the algorithms with a random test case of $N = 10^6$, $M = 10^7$.
- The runtime was measured with `time` library, and peak memory usage was measured with `tracemalloc` library in Python.

Result

We can compare the challenge results in Table 2. The running time of the Kruskal algorithm was the smallest among the three algorithms. Prim with binary heap was the slowest, and the Prim with Fibonacci heap was twice as fast as that. The memory consumption was almost the same for all three algorithms. Prim with Fibonacci heap used slightly more memeory than the other two.

4 Conclusion

The time complexity experiment and scaling challenge showed the characteristics of Kruskal and Prim algorithms, and the differences made by the kind of heap we use in implementation. I could observe that Kruskal is faster in sparse graphs as it uses a simpler data structure, but becomes less efficient in dense graphs. Prim algorithm with Fibonacci heap is very effective in large and dense graphs, both theoretically and practically. However, the 'constants' of time complexity is larger than other two due to the complex structure and operations in Fibonacci heap. Therefore, it would be more reasonable to use binary heap or Kruskal algorithm when the graph is small and sparse enough.